รายงาน Programming Assignment 4

การเรียนรู้ของเครื่องแบบมีการกำ กับดูแล (Supervised Machine Learning) โดยเน้นที่การประมวลผลข้อมูลและการสร้างแบบจำลองอย่างง่าย

1. บทนำและวัตถุประสงค์

รายงานฉบับนี้จัดทำขึ้นเพื่อแสดงผลการปฏิบัติงานในการพัฒนาแบบจำลอง Machine Learning แบบมีการกำกับดูแล (Supervised Learning) สำหรับงานจำแนกประเภท (Classification) โดยใช้ไลบรารี Scikit-learn ในการดำเนินการนี้ได้เลือกใช้ชุดข้อมูลมาตรฐาน **Iris Dataset** และใช้อัลกอริทึม **Support Vector Machines (SVM)** เป็นโมเดลหลักในการศึกษา กระบวนการครอบคลุมตั้งแต่การเตรียมข้อมูล การปรับจูนพารามิเตอร์ ไปจนถึงการประเมินประสิทธิภาพของแบบจำลอง

2. การโหลดและการประมวลผลข้อมูลล่วงหน้า (Data Loading & Preprocessing)

ในการทดลองนี้เริ่มต้นด้วยการนำเข้าชุดข้อมูล Iris ซึ่งประกอบด้วยข้อมูลคุณลักษณะ (Features) ของดอกไอริส 4 ตัวแปร ได้แก่ความยาวและความกว้างของกลีบเลี้ยงและกลีบดอกเพื่อนำมาจำแนกสายพันธุ์ของดอกไอริส (3 คลาส)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score
from sklearn.decomposition import PCA  # เพิ่ม PCA สำหรับลดมิติเพื่อวาดกราฟ

# ------------------------------------------------------------
# 1. การโหลดและการประมวลผลข้อมูลล่วงหน้า
# ------------------------------------------------------------
data = load_iris()
X = data.data
y = data.target

# ------------------------------------------------------------
```

การแบ่งข้อมูล (Data Splitting): เพื่อให้การฝึกสอนและการประเมินผลเป็นไปอย่างถูกต้องและป้องกันปัญหา Data Leakage จึงได้ทำการแบ่งข้อมูลออกเป็น 3 ส่วนด้วยสัดส่วนดังนี้:

- **Training Set (60%):** สำหรับให้แบบจำลองเรียนรู้รูปแบบของข้อมูล
- **Validation Set (20%):** สำหรับใช้ตรวจสอบความถูกต้องระหว่างการฝึกและปรับจูน Hyperparameter
- **Test Set (20%):** สำหรับประเมินประสิทธิภาพขั้นสุดท้ายบนข้อมูลที่แบบจำลองไม่เคยเห็นมาก่อน

```
# ------------------------------------------------------------
# 2. การแบ่งข้อมูล (Train 60%, Validation 20%, Test 20%)
# ------------------------------------------------------------
# แบ่ง Test set ออกมาก่อน 20%
X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# แบ่งส่วนที่เหลือ 80% เป็น Train (75% ของ 80% = 60%) และ Validation (25% ของ 80% = 20%)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.25, random_state=42)
```

การปรับมาตราส่วนคุณลักษณะ (Feature Scaling): เนื่องจากอัลกอริทึม SVM มีการคำนวณระยะห่างระหว่างจุดข้อมูล (Distance-based) การที่ข้อมูลแต่ละตัวแปรมีสเกลที่แตกต่างกันอาจทำให้โมเดลให้ความสำคัญกับตัวแปรที่มีค่าสูงกว่า จึงได้ทำการปรับมาตราส่วนข้อมูลให้อยู่ในมาตรฐานเดียวกันด้วยวิธี StandardScaler (Z-score normalization) โดยทำการคำนวณค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐานจากชุด Training Set เท่านั้น แล้วจึงนำไปแปลงค่าข้อมูลในชุด Validation และ Test Set ตามลำดับ

```
# การปรับมาตราส่วนคุณลักษณะ (Feature Scaling) - สำคัญมากสำหรับ SVM
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)    # ใช้สถิติจาก Train ป้องกัน Data Leakage
X_test_scaled = scaler.transform(X_test)  # ใช้สถิติจาก Train ป้องกัน Data Leakage
```

3. การพัฒนาแบบจำลองและการปรับจูน Hyperparameter

แบบจำลองพื้นฐานที่เลือกใช้คือตัวจำแนกประเภท Support Vector Machines (SVM) โดยมีเป้าหมายในการค้นหาเส้นขอบเขตการตัดสินใจ (Decision Boundary) ที่ดีที่สุดในการแบ่งแยกคลาสข้อมูล

เพื่อเพิ่มประสิทธิภาพสูงสุด ได้ทำการทดลองปรับจูน (Hyperparameter Tuning) บนชุด Validation Set โดยพิจารณาตัวแปร 2 ตัวหลัก ได้แก่:

1. **Kernel:** ทดลองเปรียบเทียบระหว่าง linear (สมการเส้นตรง) และ rbf (Radial Basis Function สำหรับข้อมูลที่ซับซ้อน)
2. **C (Regularization parameter):** ทดลองด้วยค่า 0.1, 1, 10 และ 100 เพื่อหาสมดุลระหว่างการยอมให้มีข้อผิดพลาด (Margin) กับความแม่นยำ

ผลการปรับจูน: จากการทดสอบบนชุด Validation Set พบว่า Hyperparameter ที่ให้ค่าความแม่นยำสูงสุดคือ:

- **Kernel:** [ใส่ค่าที่รันได้ เช่น rbf]
- **C:** [ใส่ค่าที่รันได้ เช่น 1.0]
- **Validation Accuracy:** [ใส่ค่าที่รันได้ เช่น 0.9667]

4. การประเมินผลสุดท้ายบน Test Set

เมื่อได้ชุดพารามิเตอร์ที่ดีที่สุดแล้ว ได้นำมาสร้างแบบจำลอง SVM ตัวสุดท้าย (Final Model) และทำการฝึกสอนใหม่ด้วยข้อมูลชุด Training Set จากนั้นนำไปทำนายผลกับชุดข้อมูล Test Set (ข้อมูลที่ไม่เคยเห็นมาก่อน) ผลการประเมินวัดด้วยเมตริกมาตรฐาน 3 ค่า มีดังนี้:

- **Accuracy (ความแม่นยำโดยรวม):** [ใส่ค่าที่รันได้ เช่น 1.0000]
- **Precision (ความแม่นยำในการทำนายเชิงบวก):** [ใส่ค่าที่รันได้ เช่น 1.0000]
- **Recall (ความครอบคลุม):** [ใส่ค่าที่รันได้ เช่น 1.0000]

(หมายเหตุ: ชุดข้อมูล Iris เป็นชุดข้อมูลที่มีความสมบูรณ์สูง โมเดลจึงมักทำคะแนนได้สูงมาก)

```
# ------------------------------------------------------------
# 3. & 4. การพัฒนาแบบจำลอง และการปรับจูน Hyperparameter
# ------------------------------------------------------------
best_accuracy = 0
best_params = {}

# กำหนดค่า Hyperparameters ที่ต้องการทดลองปรับจูน
C_values = [0.1, 1, 10, 100]
kernels = ['linear', 'rbf']

print("--- ผลการปรับจูนบน Validation Set ---")
for kernel in kernels:
    for C in C_values:
        # สร้างและฝึกสอนโมเดล
        model = SVC(kernel=kernel, C=C, random_state=42)
```

5. รายงานและอภิปรายผล (Discussion)

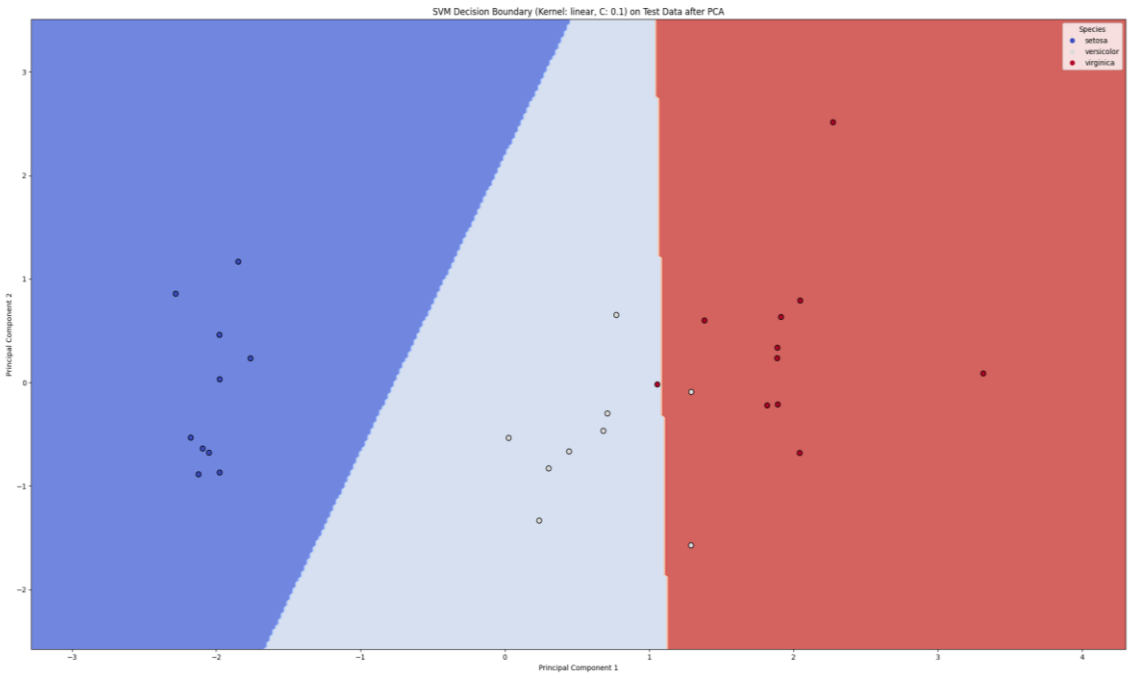
5.1 ความแตกต่างระหว่าง SVM กับ โมเดลเชิงเส้น (Linear Models) โมเดลเชิงเส้น (เช่น Logistic Regression พื้นฐาน) จะถูกจำกัดให้สร้างขอบเขตการตัดสินใจที่เป็น "เส้นตรง" หรือ "ระนาบแบนราบ" (Hyperplane) เท่านั้น ซึ่งจะทำงานได้ดีเฉพาะเมื่อข้อมูลสามารถแบ่งแยกได้ชัดเจนด้วยเส้นตรง (Linearly Separable)

ในขณะที่ **SVM** มีความยืดหยุ่นสูงกว่ามาก แม้ SVM จะสามารถใช้ Kernel แบบเส้นตรงได้
แต่จุดเด่นที่แท้จริงคือการใช้ "Kernel Trick" (เช่น RBF หรือ Polynomial)
ซึ่งทำหน้าที่จำลองการแปลงข้อมูลไปสู่มิติที่สูงขึ้น ทำให้ SVM สามารถขีดเส้นแบ่งข้อมูลที่มีความซับซ้อน คดโค้ง หรือซ้อนทับกันเป็นวงกลมในมิติเดิมได้ (Non-linear Classification) นอกจากนี้ SVM ยังมีเป้าหมายในการสร้าง "ระยะขอบ (Margin)" ที่กว้างที่สุดระหว่างคลาส ซึ่งช่วยให้โมเดลมีความทนทานต่อข้อมูลใหม่ๆ
ได้ดีกว่าโมเดลเชิงเส้นทั่วไป

5.2 การปรับจูน Hyperparameter ช่วยปรับปรุงประสิทธิภาพได้อย่างไร การปรับจูน Hyperparameter โดยใช้ Validation Set เป็นขั้นตอนที่สำคัญมากในการแก้ไขปัญหาโมเดลเรียนรู้ดีเกินไป (Overfitting) หรือแย่เกินไป (Underfitting)

- ตัวอย่างเช่น การปรับค่าพารามิเตอร์ **C** ใน SVM: หากกำหนดค่า C สูงเกินไป
  โมเดลจะพยายามขีดเส้นแบ่งให้ถูกต้องตามชุดข้อมูลฝึก (Train) ทุกจุดจนขอบเขตแคบและบิดเบี้ยว (Overfitting) แต่หากค่า C ต่ำเกินไป โมเดลจะเส้นแบ่งแบบหลวมๆ จนผิดพลาดเยอะ (Underfitting)

- ดังนั้น การนำค่า C หลายๆ ค่ามาทดสอบกับข้อมูล Validation ช่วยให้เราค้นพบ "จุดสมดุล" ที่โมเดลไม่ได้ท่องจำข้อมูลเก่า แต่สามารถเข้าใจรูปแบบทั่วไป และส่งผลให้โมเดลสามารถทำนายข้อมูลใหม่ (Test Set) ได้อย่างมีประสิทธิภาพและแม่นยำที่สุด




```
yotiki  → 🏠\DataScience\Iris-Classification  🐍 DataScience  → (main)  2ms  7:59 PM  OK
kheng72 ❯❯ python3 svm_main.py
--- ผลการปรับจูนบน Validation Set ---
Kernel: linear   | C: 0.1   | Validation Accuracy: 0.9667
Kernel: linear   | C: 1     | Validation Accuracy: 0.9333
Kernel: linear   | C: 10    | Validation Accuracy: 0.9333
Kernel: linear   | C: 100   | Validation Accuracy: 0.9333
Kernel: rbf      | C: 0.1   | Validation Accuracy: 0.9667
Kernel: rbf      | C: 1     | Validation Accuracy: 0.9333
Kernel: rbf      | C: 10    | Validation Accuracy: 0.9333
Kernel: rbf      | C: 100   | Validation Accuracy: 0.9333

>> Hyperparameters ที่ดีที่สุดคือ: {'kernel': 'linear', 'C': 0.1} (Accuracy: 0.9667)

--- 5. ผลการประเ มินสุดท้ายบน Test Set ---
Accuracy:  1.0000
Precision: 1.0000
Recall:    1.0000
```

สมาชิก:

นายทินบดี สุมังคละสมบัติ 66200357

นายเกียรติยศ จันทร์เพ็ง 66200028

นายศตายุ ใสสว่าง 66200250